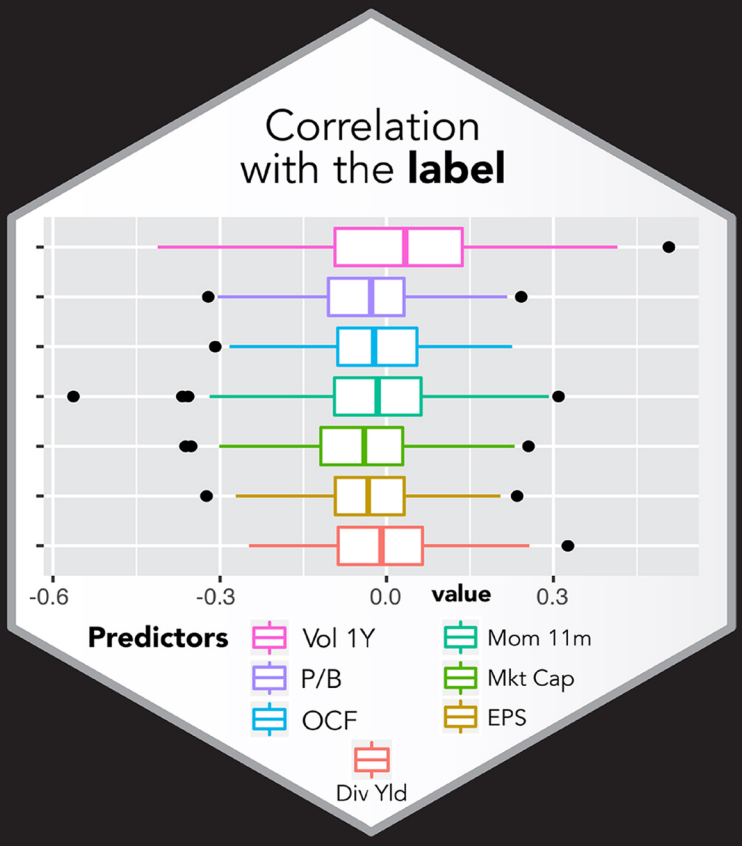
Guillaume Coqueret
Tony Guida

Machine Learning for Factor Investing

R Version



Chapman & Hall/CRC FINANCIAL MATHEMATICS SERIES

Machine Learning for Factor Investing

CHAPMAN & HALL/CRC
Financial Mathematics Series

Aims and scope:
The field of financial mathematics forms an ever-expanding slice of the financial sector. This series aims to capture new developments and summarize what is known over the whole spectrum of this field. It will include a broad range of textbooks, reference works and handbooks that are meant to appeal to both academics and practitioners. The inclusion of numerical code and concrete real-world examples is highly encouraged.

Series Editors
M.A.H. Dempster
Centre for Financial Research
Department of Pure Mathematics and Statistics
University of Cambridge

Dilip B. Madan
Robert H. Smith School of Business
University of Maryland

Rama Cont
Department of Mathematics
Imperial College

Metamodeling for Variable Annuities
Guojun Gan and Emiliano A. Valdez

Modeling Fixed Income Securities and Interest Rate Options
Robert A. Jarrow

Financial Modelling in Commodity Markets
Viviana Fanelli

Introductory Mathematical Analysis for Quantitative Finance
Daniele Ritelli, Giulia Spaletta

Handbook of Financial Risk Management
Thierry Roncalli

Optional Processes
Stochastic Calculus and Applications
Mohamed Abdelghani, Alexander Melnikov

Machine Learning for Factor Investing
R Version
Guillaume Coqueret and Tony Guida

For more information about this series please visit: *https://www.crcpress.com/Chapman-and-HallCRC-Financial-Mathematics-Series/book-series/CHFINANCMTH*

Machine Learning for Factor Investing

R Version

Guillaume Coqueret and Tony Guida

CRC Press
Taylor & Francis Group
Boca Raton London New York

CRC Press is an imprint of the
Taylor & Francis Group, an **informa** business
A CHAPMAN & HALL BOOK

First edition published 2021
by CRC Press
6000 Broken Sound Parkway NW, Suite 300, Boca Raton, FL 33487-2742

and by CRC Press
2 Park Square, Milton Park, Abingdon, Oxon, OX14 4RN

© 2021 Taylor & Francis Group, LLC

CRC Press is an imprint of Taylor & Francis Group, LLC

Reasonable efforts have been made to publish reliable data and information, but the author and publisher cannot assume responsibility for the validity of all materials or the consequences of their use. The authors and publishers have attempted to trace the copyright holders of all material reproduced in this publication and apologize to copyright holders if permission to publish in this form has not been obtained. If any copyright material has not been acknowledged please write and let us know so we may rectify in any future reprint.

Except as permitted under U.S. Copyright Law, no part of this book may be reprinted, reproduced, transmitted, or utilized in any form by any electronic, mechanical, or other means, now known or hereafter invented, including photocopying, microfilming, and recording, or in any information storage or retrieval system, without written permission from the publishers.

For permission to photocopy or use material electronically from this work, access www.copyright.com or contact the Copyright Clearance Center, Inc. (CCC), 222 Rosewood Drive, Danvers, MA 01923, 978-750-8400. For works that are not available on CCC please contact mpkbookspermissions@tandf.co.uk

Trademark notice: Product or corporate names may be trademarks or registered trademarks, and are used only for identification and explanation without intent to infringe.

Library of Congress Cataloging-in-Publication Data

ISBN: 9780367473228 (hbk)
ISBN: 9780367545864 (pbk)
ISBN: 9781003034858 (ebk)

To Leslie and Selin.



Taylor & Francis
Taylor & Francis Group
http://taylorandfrancis.com

Contents

Preface **xiii**

I Introduction **1**

1 Notations and data **3**
1.1 Notations . . . 3
1.2 Dataset . . . 4

2 Introduction **9**
2.1 Context . . . 9
2.2 Portfolio construction: the workflow . . . 10
2.3 Machine learning is no magic wand . . . 11

3 Factor investing and asset pricing anomalies **13**
3.1 Introduction . . . 14
3.2 Detecting anomalies . . . 15
3.2.1 Challenges . . . 15
3.2.2 Simple portfolio sorts . . . 15
3.2.3 Factors . . . 17
3.2.4 Fama-Macbeth regressions . . . 22
3.2.5 Factor competition . . . 25
3.2.6 Advanced techniques . . . 26
3.3 Factors or characteristics? . . . 27
3.4 Hot topics: momentum, timing and ESG . . . 28
3.4.1 Factor momentum . . . 28
3.4.2 Factor timing . . . 29
3.4.3 The green factors . . . 30
3.5 The links with machine learning . . . 30
3.5.1 A short list of recent references . . . 31
3.5.2 Explicit connections with asset pricing models . . . 31
3.6 Coding exercises . . . 34

4 Data preprocessing **35**
4.1 Know your data . . . 35
4.2 Missing data . . . 38
4.3 Outlier detection . . . 40
4.4 Feature engineering . . . 41
4.4.1 Feature selection . . . 41
4.4.2 Scaling the predictors . . . 41
4.5 Labelling . . . 42
4.5.1 Simple labels . . . 42
4.5.2 Categorical labels . . . 43

4.5.3 The triple barrier method . . . 44
4.5.4 Filtering the sample . . . 45
4.5.5 Return horizons . . . 46
4.6 Handling persistence . . . 47
4.7 Extensions . . . 47
4.7.1 Transforming features . . . 47
4.7.2 Macro-economic variables . . . 48
4.7.3 Active learning . . . 48
4.8 Additional code and results . . . 50
4.8.1 Impact of rescaling: graphical representation . . . 50
4.8.2 Impact of rescaling: toy example . . . 52
4.9 Coding exercises . . . 53

II Common supervised algorithms 55

5 Penalized regressions and sparse hedging for minimum variance portfolios 57
5.1 Penalized regressions . . . 57
5.1.1 Simple regressions . . . 57
5.1.2 Forms of penalizations . . . 58
5.1.3 Illustrations . . . 60
5.2 Sparse hedging for minimum variance portfolios . . . 62
5.2.1 Presentation and derivations . . . 62
5.2.2 Example . . . 65
5.3 Predictive regressions . . . 67
5.3.1 Literature review and principle . . . 67
5.3.2 Code and results . . . 68
5.4 Coding exercise . . . 68

6 Tree-based methods 69
6.1 Simple trees . . . 69
6.1.1 Principle . . . 69
6.1.2 Further details on classification . . . 71
6.1.3 Pruning criteria . . . 72
6.1.4 Code and interpretation . . . 73
6.2 Random forests . . . 76
6.2.1 Principle . . . 76
6.2.2 Code and results . . . 78
6.3 Boosted trees: Adaboost . . . 79
6.3.1 Methodology . . . 79
6.3.2 Illustration . . . 82
6.4 Boosted trees: extreme gradient boosting . . . 82
6.4.1 Managing loss . . . 83
6.4.2 Penalization . . . 83
6.4.3 Aggregation . . . 84
6.4.4 Tree structure . . . 85
6.4.5 Extensions . . . 86
6.4.6 Code and results . . . 86
6.4.7 Instance weighting . . . 88
6.5 Discussion . . . 89
6.6 Coding exercises . . . 90

7 Neural networks **91**
7.1 The original perceptron . . . 92
7.2 Multilayer perceptron . . . 93
7.2.1 Introduction and notations . . . 93
7.2.2 Universal approximation . . . 96
7.2.3 Learning via back-propagation . . . 97
7.2.4 Further details on classification . . . 100
7.3 How deep we should go and other practical issues . . . 101
7.3.1 Architectural choices . . . 101
7.3.2 Frequency of weight updates and learning duration . . . 102
7.3.3 Penalizations and dropout . . . 103
7.4 Code samples and comments for vanilla MLP . . . 104
7.4.1 Regression example . . . 104
7.4.2 Classification example . . . 107
7.4.3 Custom losses . . . 111
7.5 Recurrent networks . . . 112
7.5.1 Presentation . . . 112
7.5.2 Code and results . . . 114
7.6 Other common architectures . . . 117
7.6.1 Generative adversarial networks . . . 117
7.6.2 Autoencoders . . . 118
7.6.3 A word on convolutional networks . . . 119
7.6.4 Advanced architectures . . . 121
7.7 Coding exercise . . . 121

8 Support vector machines **123**
8.1 SVM for classification . . . 123
8.2 SVM for regression . . . 126
8.3 Practice . . . 127
8.4 Coding exercises . . . 128

9 Bayesian methods **129**
9.1 The Bayesian framework . . . 129
9.2 Bayesian sampling . . . 131
9.2.1 Gibbs sampling . . . 131
9.2.2 Metropolis-Hastings sampling . . . 131
9.3 Bayesian linear regression . . . 132
9.4 Naive Bayes classifier . . . 135
9.5 Bayesian additive trees . . . 138
9.5.1 General formulation . . . 138
9.5.2 Priors . . . 138
9.5.3 Sampling and predictions . . . 139
9.5.4 Code . . . 141

III From predictions to portfolios **143**

10 Validating and tuning **145**
10.1 Learning metrics . . . 145
10.1.1 Regression analysis . . . 145
10.1.2 Classification analysis . . . 147
10.2 Validation . . . 151

10.2.1 The variance-bias tradeoff: theory . . . 151
10.2.2 The variance-bias tradeoff: illustration . . . 154
10.2.3 The risk of overfitting: principle . . . 156
10.2.4 The risk of overfitting: some solutions . . . 157
10.3 The search for good hyperparameters . . . 158
10.3.1 Methods . . . 158
10.3.2 Example: grid search . . . 160
10.3.3 Example: Bayesian optimization . . . 162
10.4 Short discussion on validation in backtests . . . 163

11 Ensemble models **165**
11.1 Linear ensembles . . . 166
11.1.1 Principles . . . 166
11.1.2 Example . . . 167
11.2 Stacked ensembles . . . 170
11.2.1 Two-stage training . . . 170
11.2.2 Code and results . . . 170
11.3 Extensions . . . 172
11.3.1 Exogenous variables . . . 172
11.3.2 Shrinking inter-model correlations . . . 173
11.4 Exercise . . . 176

12 Portfolio backtesting **177**
12.1 Setting the protocol . . . 177
12.2 Turning signals into portfolio weights . . . 179
12.3 Performance metrics . . . 181
12.3.1 Discussion . . . 181
12.3.2 Pure performance and risk indicators . . . 182
12.3.3 Factor-based evaluation . . . 183
12.3.4 Risk-adjusted measures . . . 184
12.3.5 Transaction costs and turnover . . . 184
12.4 Common errors and issues . . . 185
12.4.1 Forward looking data . . . 185
12.4.2 Backtest overfitting . . . 185
12.4.3 Simple safeguards . . . 187
12.5 Implication of non-stationarity: forecasting is hard . . . 187
12.5.1 General comments . . . 187
12.5.2 The no free lunch theorem . . . 188
12.6 First example: a complete backtest . . . 189
12.7 Second example: backtest overfitting . . . 193
12.8 Coding exercises . . . 196

IV Further important topics **197**

13 Interpretability **199**
13.1 Global interpretations . . . 200
13.1.1 Simple models as surrogates . . . 200
13.1.2 Variable importance (tree-based) . . . 201
13.1.3 Variable importance (agnostic) . . . 203
13.1.4 Partial dependence plot . . . 205
13.2 Local interpretations . . . 206

13.2.1 LIME . . . 207
13.2.2 Shapley values . . . 210
13.2.3 Breakdown . . . 212

14 Two key concepts: causality and non-stationarity 215
14.1 Causality . . . 216
14.1.1 Granger causality . . . 216
14.1.2 Causal additive models . . . 217
14.1.3 Structural time series models . . . 220
14.2 Dealing with changing environments . . . 223
14.2.1 Non-stationarity: yet another illustration . . . 225
14.2.2 Online learning . . . 227
14.2.3 Homogeneous transfer learning . . . 229

15 Unsupervised learning 233
15.1 The problem with correlated predictors . . . 233
15.2 Principal component analysis and autoencoders . . . 235
15.2.1 A bit of algebra . . . 236
15.2.2 PCA . . . 236
15.2.3 Autoencoders . . . 239
15.2.4 Application . . . 240
15.3 Clustering via k-means . . . 241
15.4 Nearest neighbors . . . 243
15.5 Coding exercise . . . 245

16 Reinforcement learning 247
16.1 Theoretical layout . . . 247
16.1.1 General framework . . . 247
16.1.2 Q-learning . . . 249
16.1.3 SARSA . . . 251
16.2 The curse of dimensionality . . . 252
16.3 Policy gradient . . . 253
16.3.1 Principle . . . 253
16.3.2 Extensions . . . 254
16.4 Simple examples . . . 256
16.4.1 Q-learning with simulations . . . 256
16.4.2 Q-learning with market data . . . 257
16.5 Concluding remarks . . . 259
16.6 Exercises . . . 260

V Appendix 261

17 Data description 263

18 Solutions to exercises 267
18.1 Chapter 4 . . . 267
18.2 Chapter 5 . . . 269
18.3 Chapter 6 . . . 273
18.4 Chapter 7 . . . 273
18.5 Chapter 8: the autoencoder model . . . 276
18.6 Chapter 9 . . . 278
18.7 Chapter 12: ensemble neural network . . . 279

18.8 Chapter 13 281
18.8.1 EW portfolios with the tidyverse 281
18.8.2 Advanced weighting function 282
18.8.3 Functional programming in the backtest 283
18.9 Chapter 16 285
18.10 Chapter 17 285

Bibliography **289**

Index **319**

Preface

This book is intended to cover some advanced modelling techniques applied to equity **investment strategies** that are built on **firm characteristics**. The content is threefold. First, we try to simply explain the ideas behind most mainstream machine learning algorithms that are used in equity asset allocation. Second, we mention a wide range of academic references for the readers who wish to push a little further. Finally, we provide hands-on **R** code samples that show how to apply the concepts and tools on a realistic dataset which we share to encourage **reproducibility**.

What this book is not about

This book deals with machine learning (ML) tools and their applications in factor investing. Factor investing is a subfield of a large discipline that encompasses asset allocation, quantitative trading and wealth management. Its premise is that differences in the returns of firms can be explained by the characteristics of these firms. Thus, it departs from traditional analyses which rely on price and volume data only, like classical portfolio theory à la Markowitz (1952), or high frequency trading. For a general and broad treatment of Machine Learning in Finance, we refer to Dixon et al. (2020).

The topics we discuss are related to other themes that will not be covered in the monograph. These themes include:

- Applications of ML in **other financial fields**, such as **fraud detection** or **credit scoring**. We refer to Ngai et al. (2011) and Baesens et al. (2015) for general purpose fraud detection, to Bhattacharyya et al. (2011) for a focus on credit cards and to Ravisankar et al. (2011) and Abbasi et al. (2012) for studies on fraudulent financial reporting. On the topic of credit scoring, Wang et al. (2011) and Brown and Mues (2012) provide overviews of methods and some empirical results. Also, we do not cover ML algorithms for data sampled at higher (daily or intraday) frequencies (microstructure models, limit order book). The chapter from Kearns and Nevmyvaka (2013) and the recent paper by Sirignano and Cont (2019) are good introductions on this topic.

- **Use cases of alternative datasets** that show how to leverage textual data from social media, satellite imagery, or credit card logs to predict sales, earning reports, and, ultimately, future returns. The literature on this topic is still emerging (see, e.g., Blank et al. (2019), Jha (2019) and Ke et al. (2019)) but will likely blossom in the near future.

- **Technical details** of machine learning tools. While we do provide some insights on specificities of some approaches (those we believe are important), the purpose of the book is not to serve as reference manual on statistical learning. We refer to Hastie et al.

(2009), Cornuejols et al. (2018) (written in French), James et al. (2013) (coded in R!) and Mohri et al. (2018) for a general treatment on the subject.[1] Moreover, Du and Swamy (2013) and Goodfellow et al. (2016) are solid monographs on neural networks particularly and Sutton and Barto (2018) provide a self-contained and comprehensive tour in reinforcement learning.

- Finally, the book does not cover methods of **natural language processing** (NLP) that can be used to evaluate sentiment which can in turn be translated into investment decisions. This topic has nonetheless been trending lately and we refer to Loughran and McDonald (2016), Cong et al. (2019a), Cong et al. (2019b) and Gentzkow et al. (2019) for recent advances on the matter.

The targeted audience

Who should read this book? This book is intended for two types of audiences. First, **postgraduate students** who wish to pursue their studies in quantitative finance with a view towards investment and asset management. The second target groups are **professionals from the money management industry** who either seek to pivot towards allocation methods that are based on machine learning or are simply interested in these new tools and want to upgrade their set of competences. To a lesser extent, the book can serve **scholars or researchers** who need a manual with a broad spectrum of references both on recent asset pricing issues and on machine learning algorithms applied to money management. While the book covers mostly common methods, it also shows how to implement more exotic models, like causal graphs (Chapter 14), Bayesian additive trees (Chapter 9), and hybrid autoencoders (Chapter 7).

The book assumes basic knowledge in **algebra** (matrix manipulation), **analysis** (function differentiation, gradients), **optimization** (first and second order conditions, dual forms), and **statistics** (distributions, moments, tests, simple estimation method like maximum likelihood). A minimal **financial culture** is also required: simple notions like stocks, accounting quantities (e.g., book value) will not be defined in this book. Lastly, all examples and illustrations are coded in R. A minimal culture of the language is sufficient to understand the code snippets which rely heavily on the most common functions of the tidyverse (Wickham et al. (2019), www.tidyverse.org), and piping (Bache and Wickham (2014), Mailund (2019)).

How this book is structured

The book is divided into four parts.

Part I gathers preparatory material and starts with notations and data presentation (Chapter 1), followed by introductory remarks (Chapter 2). Chapter 3 outlines the economic

[1] For a list of online resources, we recommend the curated page https://github.com/josephmisiti/awesome-machine-learning/blob/master/books.md.

foundations (theoretical and empirical) of factor investing and briefly sums up the dedicated recent literature. Chapter 4 deals with data preparation. It rapidly recalls the basic tips and warns about some major issues.

Part II of the book is dedicated to predictive algorithms in supervised learning. Those are the most common tools that are used to forecast financial quantities (returns, volatilities, Sharpe ratios, etc.). They range from penalized regressions (Chapter 5), to tree methods (Chapter 6), encompassing neural networks (Chapter 7), support vector machines (Chapter 8) and Bayesian approaches (Chapter 9).

The next portion of the book bridges the gap between these tools and their applications in finance. Chapter 10 details how to assess and improve the ML engines defined beforehand. Chapter 11 explains how models can be combined and often why that may not be a good idea. Finally, one of the most important chapters (Chapter 12) reviews the critical steps of portfolio backtesting and mentions the frequent mistakes that are often encountered at this stage.

The end of the book covers a range of advanced topics connected to machine learning more specifically. The first one is **interpretability**. ML models are often considered to be black boxes and this raises trust issues: how and why should one trust ML-based predictions? Chapter 13 is intended to present methods that help understand what is happening under the hood. Chapter 14 is focused on **causality**, which is both a much more powerful concept than correlation and also at the heart of many recent discussions in Artificial Intelligence (AI). Most ML tools rely on correlation-like patterns and it is important to underline the benefits of techniques related to causality. Finally, Chapters 15 and 16 are dedicated to non-supervised methods. The latter can be useful, but their financial applications should be wisely and cautiously motivated.

Companion website

This book is entirely available at `http://www.mlfactor.com`. It is important that not only the content of the book be accessible, but also the data and code that are used throughout the chapters. They can be found at `https://github.com/shokru/mlfactor.github.io/tree/master/material`. The online version of the book will be updated beyond the publication of the printed version.

Why R?

The supremacy of Python as *the* dominant ML programming language is a widespread belief. This is because almost all applications of deep learning (which is as of 2020 one of the most fashionable branches of ML) are coded in Python via Tensorflow or Pytorch. The fact is that **R** has a **lot** to offer as well. First of all, let us not forget that one of the most influencial textbooks in ML (Hastie et al. (2009)) is written by statisticians who code in R. Moreover, many statistics-orientated algorithms (e.g., BARTs in Section 9.5) are primarily

coded in R and not always in Python. The R offering in Bayesian packages in general (https://cran.r-project.org/web/views/Bayesian.html) and in Bayesian learning in particular is probably unmatched.

There are currently several ML frameworks available in R.

- **caret**: https://topepo.github.io/caret/index.html, a compilation of more than 200 ML models;
- **tidymodels**: https://github.com/tidymodels, a recent collection of packages for ML workflow (developed by Max Kuhn at RStudio, which is a token of high quality material!);
- **rtemis**: https://rtemis.netlify.com, a general purpose package for ML and visualization;
- **mlr3**: https://mlr3.mlr-org.com/index.html, also a simple framework for ML models;
- **h2o**: https://github.com/h2oai/h2o-3/tree/master/h2o-r, a large set of tools provided by h2o (coded in Java);
- **Open ML**: https://github.com/openml/openml-r, the R version of the OpenML (www.openml.org) community.

Moreover, via the *reticulate* package, it is possible (but not always easy) to benefit from Python tools as well. The most prominent example is the adaptation of the *tensorflow* and *keras* libraries to R. Thus, some very advanced Python material is readily available to R users. This is also true for other resources, like Stanford's CoreNLP library (in Java) which was adapted to R in the package *coreNLP* (which we will not use in this book).

Coding instructions

One of the purposes of the book is to propose a large-scale tutorial of ML applications in financial predictions and portfolio selection. Thus, one keyword is **REPRODUCIBILITY**! In order to duplicate our results (up to possible randomness in some learning algorithms), you will need running versions of R and RStudio on your computer. The best books to learn R are also often freely available online. A short list can be found here https://rstudio.com/resources/books/. The monograph *R for Data Science* is probably the most crucial.

In terms of coding requirements, we rely heavily on the **tidyverse**, which is a collection of **packages** (or libraries). The three packages we use most are **dplyr** which implements simple data manipulations (filter, select, arrange), **tidyr** which formats data in a tidy fashion, and **ggplot**, for graphical outputs.

A list of the packages we use can be found in Table 1 below. Packages with a star $*$ need to be installed via *bioconductor*.[2] Packages with a plus $^+$ need to be installed **manually**.[3]

TABLE 1: List of all packages used in the book.

Package	Purpose	Chapter(s)
BART	Bayesian additive trees	10
broom	Tidy regression output	5
CAM$^+$	Causal Additive Models	15
caTools	AUC curves	11
CausalImpact	Causal inference with structural time series	15
cowplot	Stacking plots	4 & 13
breakDown	Breakdown interpretability	14
dummies	One-hot encoding	8
e1071	Support Vector Machines	9
factoextra	PCA visualization	16
fastAdaboost	Boosted trees	7
forecast	Autocorrelation function	4
FNN	Nearest Neighbors detection	16
ggpubr	Combining plots	11
glmnet	Penalized regressions	6
iml	Interpretability tools	14
keras	Neural networks	8
lime	Interpretability	14
lmtest	Granger causality	15
lubridate	Handling dates	All (or many)
naivebayes	Naive Bayes classifier	10
pcalg	Causal graphs	15
quadprog	Quadratic programming	12
quantmod	Data extraction	4, 12
randomForest	Random forests	7
rBayesianOptimization	Bayesian hyperparameter tuning	11
ReinforcementLearning	Reinforcement Learning	17
Rgraphviz*	Causal graphs	15
rpart and *rpart.plot*	Simple decision trees	7
spBayes	Bayesian linear regression	10
tidyverse	Environment for data science, data wrangling	All
xgboost	Boosted trees	7
xtable	Table formatting	4

Of all of these packages (or collections thereof), the **tidyverse** and **lubridate** are compulsory in almost all sections of the book. To install a new package in R, just type

install.packages("name_of_the_package")

in the console. Sometimes, because of function name conflicts (especially with the select() function), we use the syntax package::function() to make sure the function call is from the

[2] One example: https://www.bioconductor.org/packages/release/bioc/html/Rgraphviz.html

[3] By copy-pasting the content of the package in the library folder. To get the address of the folder, execute the command *.libPaths()* in the R console.

right source. The exact version of the packages used to compile the book is listed in the "*renv.lock*" file available on the book's GitHub web page https://github.com/shokru/mlfactor.github.io. One minor comment is the following: while the functions *gather()* and *spread()* from the *dplyr* package have been superseded by *pivot_longer()* and *pivot_wider()*, we still use them because of their much more compact syntax.

As much as we could, we created short **code chunks** and commented each line whenever we felt it was useful. Comments are displayed at the end of a row and preceded with a single hastag #.

The book is constructed as a very big notebook, thus results are often presented below code chunks. They can be graphs or tables. Sometimes, they are simple numbers and are preceded with two hashtags ##. The example below illustrates this formatting.

```
1+2 # Example
```

```
## [1] 3
```

The book can be viewed as a very big tutorial. Therefore, most of the chunks depend on previously defined variables. When replicating parts of the code (via online code), please make sure that **the environment includes all relevant variables**. One best practice is to always start by running all code chunks from Chapter 1. For the exercises, we often resort to variables created in the corresponding chapters.

Acknowledgments

The core of the book was prepared for a series of lectures given by one of the authors to students of master's degrees in finance at EMLYON Business School and at the Imperial College Business School in the Spring of 2019. We are grateful to those students who asked fruitful questions and thereby contributed to improve the content of the book.

We are grateful to Bertrand Tavin and Gautier Marti for their thorough screening of the book. We also thank Eric André, Aurélie Brossard, Alban Cousin, Frédérique Girod, Philippe Huber, Jean-Michel Maeso, Javier Nogales and for friendly reviews; Christophe Dervieux for his help with bookdown; Mislav Sagovac and Vu Tran for their early feedback; John Kimmel for making this happen and Jonathan Regenstein for his availability, no matter the topic. Lastly, we are grateful for the anonymous reviews collected by John.

Future developments

Machine learning and factor investing are two immense research domains and the overlap between the two is also quite substantial and developing at a fast pace. The content of this book will always constitute a solid background, but it is naturally destined to obsolescence. Moreover, by construction, some subtopics and many references will have escaped our scrutiny. Our intent is to progressively improve the content of the book and update it with

the latest ongoing research. We will be grateful to any comment that helps correct or update the monograph. Thank you for sending your feedback directly (via pull requests) on the book's website which is hosted at https://github.com/shokru/mlfactor.github.io.



Taylor & Francis
Taylor & Francis Group
http://taylorandfrancis.com

Part I

Introduction



Taylor & Francis
Taylor & Francis Group
http://taylorandfrancis.com

1

Notations and data

1.1 Notations

This section aims at providing the formal mathematical conventions that will be used throughout the book.

Bold notations indicate vectors and matrices. We use capital letters for matrices and lower case letters for vectors. $\mathbf{v}'$ and $\mathbf{M}'$ denote the transposes of $\mathbf{v}$ and $\mathbf{M}$. $\mathbf{M} = [m]_{i,j}$, where i is the row index and j the column index.

We will work with two notations in parallel. The first one is the pure machine learning notation in which the **labels** (also called **output**, **dependent** variables or **predicted** variables) $\mathbf{y} = y_i$ are approximated by functions of features $\mathbf{X}_i = (x_{i,1}, \ldots, x_{i,K})$. The dimension of the feature matrix $\mathbf{X}$ is $I \times K$: there are I **instances**, **records**, or **observations** and each one of them has K **attributes**, **features**, **inputs**, or **predictors** which will serve as **independent** and **explanatory** variables (all these terms will be used interchangeably). Sometimes, to ease notations, we will write $\mathbf{x}_i$ for one instance (one row) of $\mathbf{X}$ or $\mathbf{x}_k$ for one (feature) column vector of $\mathbf{X}$.

The second notation type pertains to finance and will directly relate to the first. We will often work with discrete returns $r_{t,n} = p_{t,n}/p_{t-1,n} - 1$ computed from price data. Here t is the time index and n the asset index. Unless specified otherwise, the return is always computed over one period, though this period can sometimes be one month or one year. Whenever confusion might occur, we will specify other notations for returns.

In line with our previous conventions, the number of return dates will be T and the number of assets, N. The features or characteristics of assets will be denoted with $x_{t,n}^{(k)}$: it is the time-t value of the kth attribute of firm or asset n. In stacked notation, $\mathbf{x}_{t,n}$ will stand for the vector of characteristics of asset n at time t. Moreover, $\mathbf{r}_t$ stands for all returns at time t while $\mathbf{r}_n$ stands for all returns of asset n. Often, returns will play the role of the dependent variable, or label (in ML terms). For the riskless asset, we will use the notation $r_{t,f}$.

The link between the two notations will most of the time be the following. One **instance** (or **observation**) i will consist of one couple (t, n) of one particular date and one particular firm (if the data is perfectly rectangular with no missing field, $I = T \times N$). The label will usually be some performance measure of the firm computed over some future period, while the features will consist of the firm attributes at time t. Hence, the purpose of the machine learning engine in factor investing will be to determine the model that maps the time-t characteristics of firms to their future performance.

In terms of canonical matrices: $\mathbf{I}_N$ will denote the $(N \times N)$ identity matrix.

From the probabilistic literature, we employ the expectation operator $\mathbb{E}[\cdot]$ and the conditional expectation $\mathbb{E}_t[\cdot]$, where the corresponding filtration $\mathcal{F}_t$ corresponds to all information

available at time t. More precisely, $\mathbb{E}_t[\cdot] = \mathbb{E}[\cdot|\mathcal{F}_t]$. $\mathbb{V}[\cdot]$ will denote the variance operator. Depending on the context, probabilities will be written simply P, but sometimes we will use the heavier notation $\mathbb{P}$. Probability density functions (pdfs) will be denoted with lowercase letters (f) and cumulative distribution functions (cdfs) with uppercase letters (F). We will write equality in distribution as $X \overset{d}{=} Y$, which is equivalent to $F_X(z) = F_Y(z)$ for all z on the support of the variables. For a random process X_t, we say that it is **stationary** if the law of X_t is constant through time, i.e., $X_t \overset{d}{=} X_s$, where $\overset{d}{=}$ means equality in distribution.

Sometimes, asymptotic behaviors will be characterized with the usual Landau notation $o(\cdot)$ and $O(\cdot)$. The symbol $\propto$ refers to proportionality: $x \propto y$ means that x is proportional to y. With respect to derivatives, we use the standard notation $\frac{\partial}{\partial x}$ when differentiating with respect to x. We resort to the compact symbol ∇ when all derivatives are computed (gradient vector).

In equations, the left-hand side and right-hand side can be written more compactly: l.h.s. and r.h.s., respectively.

Finally, we turn to functions. We list a few below:
- $1_{\{x\}}$: the indicator function of the condition x, which is equal to one if x is true and to zero otherwise.
- $\phi(\cdot)$ and $\Phi(\cdot)$ are the standard Gaussian pdf and cdf.
- $\text{card}(\cdot) = \#(\cdot)$ are two notations for the cardinal function which evaluates the number of elements in a given set (provided as argument of the function).
- $\lfloor \cdot \rfloor$ is the integer part function.
- for a real number x, $[x]^+$ is the positive part of x, that is $\max(0, x)$.
- $\tanh(\cdot)$ is the hyperbolic tangent: $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$.
- $\text{ReLu}(\cdot)$ is the rectified linear unit: $\text{ReLu}(x) = \max(0, x)$.
- $\text{s}(\cdot)$ will be the softmax function: $s(\mathbf{x})_i = \frac{e^{x_i}}{\sum_{j=1}^{J} e^{x_j}}$, where the subscript i refers to the ith element of the vector.

1.2 Dataset

Throughout the book, and for the sake of **reproducibility**, we will illustrate the concepts we present with examples of implementation based on a single financial dataset available at https://github.com/shokru/mlfactor.github.io/tree/master/material. This dataset comprises information on 1,207 stocks listed in the US (possibly originating from Canada or Mexico). The time range starts in November 1998 and ends in March 2019. For each point in time, 93 **characteristics** describe the firms in the sample. These attributes cover a wide range of topics:

- **valuation** (earning yields, accounting ratios);
- **profitability** and quality (return on equity);
- **momentum** and technical analysis (past returns, relative strength index);
- **risk** (volatilities);

- **estimates** (earnings-per-share);
- **volume** and **liquidity** (share turnover).

The sample is not perfectly rectangular: there are no missing points, but the number of firms and their attributes is not constant through time. This makes the computations in the backtest more tricky, but also more realistic.

```
library(tidyverse)                      # Activate the data science package
library(lubridate)                      # Activate the date management package
load("data_ml.RData")                   # Load the data
data_ml <- data_ml %>%
    filter(date > "1999-12-31",         # Keep the date with sufficient data points
           date < "2019-01-01") %>%
    arrange(stock_id, date)             # Order the data
data_ml[1:6, 1:6]                       # Sample values
```

```
## # A tibble: 6 x 6
##   stock_id date       Advt_12M_Usd Advt_3M_Usd Advt_6M_Usd Asset_Turnover
##      <int> <date>            <dbl>       <dbl>       <dbl>          <dbl>
## 1        1 2000-01-31         0.41        0.39        0.42           0.19
## 2        1 2000-02-29         0.41        0.39        0.4            0.19
## 3        1 2000-03-31         0.4         0.37        0.37           0.2
## 4        1 2000-04-30         0.39        0.36        0.37           0.2
## 5        1 2000-05-31         0.4         0.42        0.4            0.2
## 6        1 2000-06-30         0.41        0.47        0.42           0.21
```

The data has 99 columns and 268336 rows. The first two columns indicate the stock identifier and the date. The next 93 columns are the features (see Table 17.1 in the Appendix for details). The last four columns are the labels. The points are sampled at the monthly frequency. As is always the case in practice, the number of assets changes with time, as is shown in Figure 1.1.

```
data_ml  %>%
    group_by(date) %>%                                  # Group by date
    summarize(nb_assets = stock_id %>%                  # Count nb assets
                  as.factor() %>% nlevels()) %>%
    ggplot(aes(x = date, y = nb_assets)) + geom_col() +  # Plot
    coord_fixed(3)
```

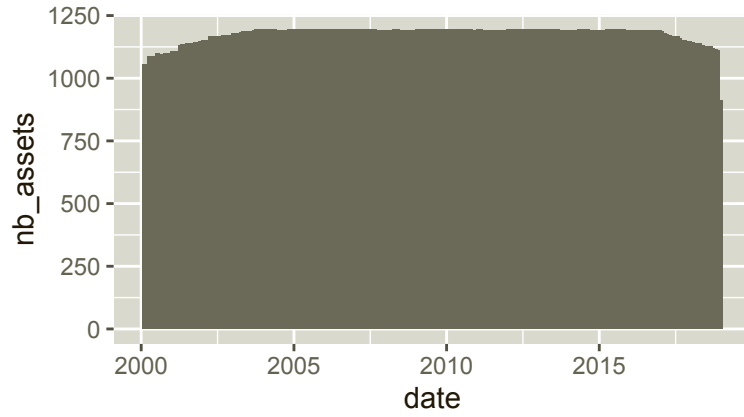


FIGURE 1.1: Number of assets through time.

There are four immediate **labels** in the dataset: R1M_Usd, R3M_Usd, R6M_Usd and R12M_Usd, which correspond to the 1-month, 3-month, 6-month and 12-month future/forward returns of the stocks. The returns are **total returns**, that is, they incorporate potential **dividend** payments over the considered periods. This is a better proxy of financial gain compared to price returns only. We refer to the analysis of Hartzmark and Solomon (2019) for a study on the impact of decoupling price returns and dividends. These labels are located in the last 4 columns of the dataset. We provide their descriptive statistics below.

```
## # A tibble: 4 x 5
##   Label      mean    sd    min   max
##   <chr>     <dbl> <dbl>  <dbl> <dbl>
## 1 R12M_Usd 0.137  0.738 -0.991  96.0
## 2 R1M_Usd  0.0127 0.176 -0.922  30.2
## 3 R3M_Usd  0.0369 0.328 -0.929  39.4
## 4 R6M_Usd  0.0723 0.527 -0.98  107.
```

In anticipation for future models, we keep the name of the predictors in memory. In addition, we also keep a much shorter list of predictors.

```
features <- colnames(data_ml[3:95]) # Keep the feature's column names (hard-coded, beware!)
features_short <- c("Div_Yld", "Eps", "Mkt_Cap_12M_Usd", "Mom_11M_Usd",
                    "Ocf", "Pb", "Vol1Y_Usd")
```

The predictors have been uniformized, that is, for any given feature and time point, the distribution is uniform. Given 1,207 stocks, the graph below cannot display a perfect rectangle.

```
data_ml %>%
    filter(date == "2000-02-29") %>%
    ggplot(aes(x = Div_Yld)) + geom_histogram(bins = 100) + coord_fixed(0.03)
```

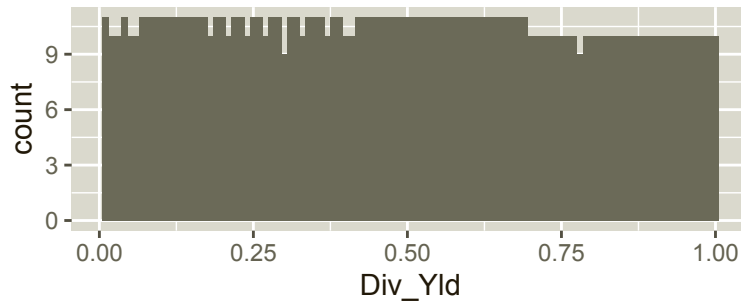


FIGURE 1.2: Distribution of the dividend yield feature on date 2000-02-29.

The original labels (future returns) are numerical and will be used for regression exercises, that is, when the objective is to predict a scalar real number. Sometimes, the exercises can be different and the purpose may be to forecast **categories** (also called **classes**), like "buy", "hold" or "sell". In order to be able to perform this type of classification analysis, we create additional labels that are categorical.

```
data_ml <- data_ml %>%
    group_by(date) %>%                                  # Group by date
    mutate(R1M_Usd_C = R1M_Usd > median(R1M_Usd),       # Create the categorical labels
           R12M_Usd_C = R1M_Usd > median(R12M_Usd)) %>%
```

```
    ungroup() %>%
    mutate_if(is.logical, as.factor)
```

The new labels are binary: they are equal to 1 (true) if the original return is above that of the median return over the considered period and to 0 (false) if not. Hence, at each point in time, half of the sample has a label equal to zero and the other half to one: some stocks overperform and others underperform.

In machine learning, models are estimated on one portion of data (**training set**) and then tested on another portion of the data (**testing set**) to assess their quality. We split our sample accordingly.

```
separation_date <- as.Date("2014-01-15")
training_sample <- filter(data_ml, date < separation_date)
testing_sample <- filter(data_ml, date >= separation_date)
```

We also keep in memory a few key variables, like the list of asset identifiers and a rectangular version of returns. For simplicity, in the computation of the latter, we shrink the investment universe to keep only the stocks for which we have the maximum number of points.

```
stock_ids <- levels(as.factor(data_ml$stock_id)) # A list of all stock_ids
stock_days <- data_ml %>%                        # Compute the number of data points per stock
    group_by(stock_id) %>% summarize(nb = n())
stock_ids_short <- stock_ids[which(stock_days$nb == max(stock_days$nb))] # Stocks with full data
returns <- data_ml %>%                           # Compute returns, in matrix format, in 3 steps:
    filter(stock_id %in% stock_ids_short) %>%    # 1. Filtering the data
    dplyr::select(date, stock_id, R1M_Usd) %>%   # 2. Keep returns along with dates & firm names
    spread(key = stock_id, value = R1M_Usd)      # 3. Put in matrix shape
```



Taylor & Francis
Taylor & Francis Group
http://taylorandfrancis.com

2

Introduction

Conclusions often echo introductions. This chapter was completed at the very end of the writing of the book. It outlines principles and ideas that are probably more relevant than the sum of technical details covered subsequently. When stuck with disappointing results, we advise the reader to take a step away from the algorithm and come back to this section to get a broader perspective of some of the issues in predictive modelling.

2.1 Context

The blossoming of machine learning in factor investing has it source at the confluence of three favorable developments: data availability, computational capacity, and economic groundings.

First, the **data**. Nowadays, classical providers, such as Bloomberg and Reuters have seen their playing field invaded by niche players and aggregation platforms.[1] In addition, high-frequency data and derivative quotes have become mainstream. Hence, firm-specific attributes are easy and often cheap to compile. This means that the size of $\mathbf{X}$ in (2.1) is now sufficiently large to be plugged into ML algorithms. The order of magnitude (in 2019) that can be reached is the following: a few hundred monthly observations over several thousand stocks (US listed at least) covering a few hundred attributes. This makes a dataset of dozens of millions of points. While it is a reasonably high figure, we highlight that the chronological depth is probably the weak point and will remain so for decades to come because accounting figures are only released on a quarterly basis. Needless to say that this drawback does not hold for high-frequency strategies.

Second, **computational power**, both through hardware and software. Storage and processing speed are not technical hurdles anymore and models can even be run on the cloud thanks to services hosted by major actors (Amazon, Microsoft, IBM and Google) and by smaller players (Rackspace, Techila). On the software side, open source has become the norm, funded by corporations (TensorFlow & Keras by Google, Pytorch by Facebook, h2o, etc.), universities (Scikit-Learn by INRIA, NLPCore by Stanford, NLTK by UPenn) and small groups of researchers (caret, xgboost, tidymodels to list but a pair of frameworks). Consequently, ML is no longer the private turf of a handful of expert computer scientists, but is on the contrary **accessible** to anyone willing to learn and code.

Finally, **economic framing**. Machine learning applications in finance were initially introduced by computer scientists and information system experts (e.g., Braun and Chandler (1987), White (1988)) and exploited shortly after by academics in financial economics (Bansal

[1] We refer to https://alternativedata.org/data-providers/[2] for a list of alternative data providers. Moreover, we recall that Quandl, an alt-data hub was acquired by Nasdaq in December 2018. As large players acquire newcomers, the field may consolidate.

and Viswanathan (1993)), and hedge funds (see, e.g., Zuckerman (2019)). Nonlinear relationships then became more mainstream in asset pricing (Freeman and Tse (1992), Bansal et al. (1993)). These contributions started to pave the way for the more brute-force approaches that have blossomed since the 2010 decade and which are mentioned throughout the book.

In the synthetic proposal of Arnott et al. (2019b), the first piece of advice is to rely on a model that makes sense economically. We agree with this stance, and the only assumption that we make in this book is that future returns depend on firm characteristics. The relationship between these features and performance is largely unknown and probably time-varying. This is why ML can be useful: to detect some hidden patterns beyond the documented asset pricing anomalies. Moreover, dynamic training allows to adapt to changing market conditions.

2.2 Portfolio construction: the workflow

Building successful portfolio strategies requires many steps. This book covers many of them but focuses predominantly on the prediction part. Indeed, allocating to assets most of the time requires to make bets and thus to presage and foresee which ones will do well and which ones will not. In this book, we mostly resort to supervised learning to forecast returns in the cross-section. The baseline equation in supervised learning,

$$\mathbf{y} = f(\mathbf{X}) + \epsilon, \tag{2.1}$$

is translated in financial terms as

$$\mathbf{r}_{t+1,n} = f(\mathbf{x}_{t,n}) + \epsilon_{t+1,n}, \tag{2.2}$$

where $f(\mathbf{x}_{t,n})$ can be viewed as the **expected return** for time $t+1$ computed at time t, that is, $\mathbb{E}_t[r_{t+1,n}]$. Note that the model is **common to all assets** (f is not indexed by n), thus it shares similarity with panel approaches.

Building accurate predictions requires to pay attention to **all** terms in the above equation. Chronologically, the first step is to gather data and to process it (see Chapter 4). To the best of our knowledge, the only consensus is that, on the $\mathbf{x}$ side, the features should include classical predictors reported in the literature: market capitalization, accounting ratios, risk measures, momentum proxies (see Chapter 3). For the dependent variable, many researchers and practitioners work with monthly returns, but other maturities may perform better out-of-sample.

While it is tempting to believe that the most crucial part is the choice of f (it is the most sophisticated, mathematically), we believe that the choice and engineering of inputs, that is, the variables, are at least as important. The usual modelling families for f are covered in Chapters 5 to 9. Finally, the errors $\epsilon_{t+1,n}$ are often overlooked. People consider that vanilla quadratic programming is the best way to go (the most common for sure!), thus the mainstream objective is to minimize squared errors. In fact, other options may be wiser choices (see for instance Section 7.4.3).

Even if the overall process, depicted in Figure 2.1, seems very sequential, it is more judicious

to conceive it as **integrated**. All steps are intertwined and each part should not be dealt with independently from the others.[3] The global framing of the problem is essential, from the choice of predictors, to the family of algorithms, not to mention the portfolio weighting schemes (see Chapter 12 for the latter).

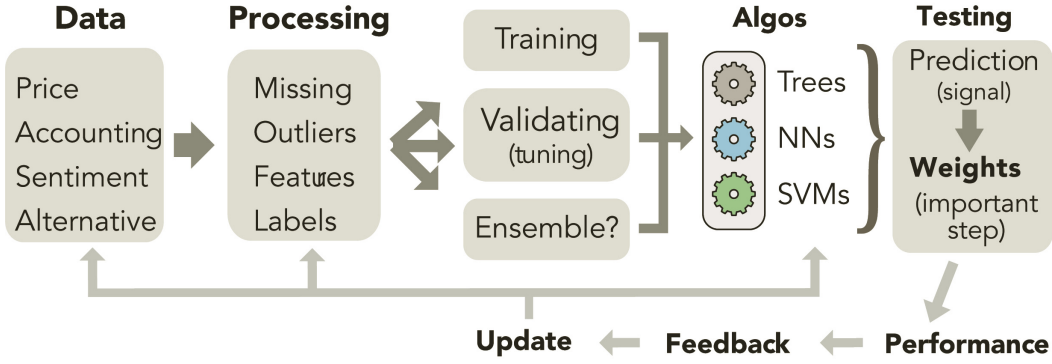


FIGURE 2.1: Simplified workflow in ML-based portfolio construction.

2.3 Machine learning is no magic wand

By definition, the curse of predictions is that they rely on **past** data to infer patterns about **subsequent** fluctuations. The more or less explicit hope of any forecaster is that the past will turn out to be a good approximation of the future. Needless to say, this is a pious wish; in general, predictions fare badly. Surprisingly, this does not depend much on the sophistication of the econometric tool. In fact, heuristic guesses are often hard to beat.

To illustrate this sad truth, the baseline algorithms that we detail in Chapters 5 to 7 yield at best mediocre results. This is done **on purpose**. This forces the reader to understand that blindly feeding data and parameters to a coded function will seldom suffice to reach satisfactory out-of-sample accuracy.

Below, we sum up some key points that we have learned through our exploratory journey in financial ML.

- The first point is that **causality** is key. If one is able to identify $X \rightarrow y$, where y are expected returns, then the problem is solved. Unfortunately, causality is incredibly hard to uncover.

- Thus, researchers have most of the time to make do with simple **correlation** patterns, which are far less informative and robust.

- Relatedly, financial datasets are extremely noisy. It is a daunting task to **extract signals** out of them. **No-arbitrage** reasonings imply that if a simple pattern yielded durable profits, it would mechanically and rapidly vanish.

[3] Other approaches are nonetheless possible, as is advocated in de Prado and Fabozzi (2020).

- The no-free lunch theorem of Wolpert (1992a) imposes that the analyst formulates views on the model. This is why economic or **econometric framing** is key. The assumptions and choices that are made regarding both the dependent variables and the explanatory features are decisive. As a corollary, data is key. The inputs given to the models are probably much more important than the choice of the model itself.

- To maximize out-of-sample efficiency, the right question is probably to paraphrase Jeff Bezos: what's not going to change? **Persistent** series are more likely to unveil enduring patterns.

- Everybody makes mistakes. Errors in loops or variable indexing are part of the journey. What matters is to **learn** from those lapses.

To conclude, we remind the reader of this obvious truth: nothing will ever replace **practice**. Gathering and cleaning data, coding backtests, tuning ML models, testing weighting schemes, debugging, starting all over again: these are all absolutely indispensable steps and tasks that must be repeated indefinitely. There is no sustitute to experience.

3

Factor investing and asset pricing anomalies

Asset pricing anomalies are the foundations of **factor investing**. In this chapter our aim is twofold:

- present simple ideas and concepts: basic factor models and common empirical facts (time-varying nature of returns and risk premia);
- provide the reader with lists of articles that go much deeper to stimulate and satisfy curiosity.

The purpose of this chapter is not to provide a full treatment of the many topics related to factor investing. Rather, it is intended to give a broad overview and cover the essential themes so that the reader is guided towards the relevant references. As such, it can serve as a short, non-exhaustive, review of the literature. The subject of factor modelling in finance is incredibly vast and the number of papers dedicated to it is substantial and still rapidly increasing.

The universe of peer-reviewed financial journals can be split in two. The first kind is the **academic** journals. Their articles are mostly written by professors, and the audience consists mostly of scholars. The articles are long and often technical. Prominent examples are the *Journal of Finance*, the *Review of Financial Studies* and the *Journal of Financial Economics*. The second type is more **practitioner**-orientated. The papers are shorter, easier to read, and target finance professionals predominantly. Two emblematic examples are the *Journal of Portfolio Management* and the *Financial Analysts Journal*. This chapter reviews and mentions articles published essentially in the first family of journals.

Beyond academic articles, several monographs are already dedicated to the topic of style allocation (a synonym of factor investing used for instance in theoretical articles (Barberis and Shleifer (2003)) or practitioner papers (Asness et al. (2015))). To cite but a few, we mention:

- Ilmanen (2011): an exhaustive excursion into risk premia, across many asset classes, with a large spectrum of descriptive statistics (across factors and periods),
- Ang (2014): covers factor investing with a strong focus on the money management industry,
- Bali et al. (2016): very complete book on the cross-section of signals with statistical analyses (univariate metrics, correlations, persistence, etc.),
- Jurczenko (2017): a tour on various topics given by field experts (factor purity, predictability, selection versus weighting, factor timing, etc.).

Finally, we mention a few wide-scope papers on this topic: Goyal (2012), Cazalet and Roncalli (2014) and Baz et al. (2015).

3.1 Introduction

The topic of factor investing, though a decades-old academic theme, has gained traction concurrently with the rise of equity traded funds (ETFs) as vectors of investment. Both have gathered momentum in the 2010 decade. Not so surprisingly, the feedback loop between practical financial engineering and academic research has stimulated both sides in a mutually beneficial manner. Practitioners rely on key scholarly findings (e.g., asset pricing anomalies) while researchers dig deeper into pragmatic topics (e.g., factor exposure or transaction costs). Recently, researchers have also tried to quantify and qualify the impact of factor indices on financial markets. For instance, Krkoska and Schenk-Hoppé (2019) analyze herding behaviors while Cong and Xu (2019) show that the introduction of composite securities increases volatility and cross-asset correlations.

The core aim of factor models is to understand the **drivers of asset prices**. Broadly speaking, the rationale behind factor investing is that the financial performance of firms depends on factors, whether they be latent and unobservable, or related to intrinsic characteristics (like accounting ratios for instance). Indeed, as Cochrane (2011) frames it, the first essential question is *which characteristics really provide independent information about average returns?* Answering this question helps understand the cross-section of returns and may open the door to their prediction.

Theoretically, linear factor models can be viewed as special cases of the arbitrage pricing theory (APT) of Ross (1976), which assumes that the return of an asset n can be modelled as a linear combination of underlying factors f_k:

$$r_{t,n} = \alpha_n + \sum_{k=1}^{K} \beta_{n,k} f_{t,k} + \epsilon_{t,n}, \tag{3.1}$$

where the usual econometric constraints on linear models hold: $\mathbb{E}[\epsilon_{t,n}] = 0$, $\text{cov}(\epsilon_{t,n}, \epsilon_{t,m}) = 0$ for $n \neq m$ and $\text{cov}(\mathbf{f}_n, \boldsymbol{\epsilon}_n) = 0$. If such factors do exist, then they are in contradiction with the cornerstone model in asset pricing: the capital asset pricing model (CAPM) of Sharpe (1964), Lintner (1965) and Mossin (1966). Indeed, according to the CAPM, the only driver of returns is the market portfolio. This explains why factors are also called 'anomalies'.

Empirical evidence of asset pricing anomalies has accumulated since the dual publication of Fama and French (1992) and Fama and French (1993). This seminal work has paved the way for a blossoming stream of literature that has its meta-studies (e.g., Green et al. (2013), Harvey et al. (2016) and McLean and Pontiff (2016)). The regression (3.1) can be evaluated once (unconditionally) or sequentially over different time frames. In the latter case, the parameters (coefficient estimates) change and the models are thus called *conditional* (we refer to Ang and Kristensen (2012) and to Cooper and Maio (2019) for recent results on this topic as well as for a detailed review on the related research). Conditional models are more flexible because they acknowledge that the drivers of asset prices may not be constant, which seems like a reasonable postulate.